Zeroing Offset Target Generator

Build specification for a single-file web app. Reference output and reference implementation are appended to this document.

1. What to build

A single self-contained HTML file that generates printable rifle zeroing targets as PDFs. The user maintains an inventory of firearms; for each one they pick the distance they will actually be shooting at and the distance they want to be zeroed for, and the app produces a print-exact target showing where to aim and where the group must land.

The premise: at a short shooting distance the bullet has not yet crossed the line of sight, so it prints low. If you deliberately zero to that offset, you are on the trajectory that crosses line of sight at your intended distance. A 10-yard target can set up a 100-yard zero.

Hard constraints

- **One file.** Everything in a single .html the user can double-click. No build step, no bundler, no server, no npm, no framework.
- **jsPDF from CDN** for PDF generation. Do not use browser print CSS — print scaling silently destroys the entire premise of this tool.
- **Vanilla JS.** No React, no Vue, no Tailwind CDN.
- **Offline-capable after first load.** No API calls, no telemetry, no accounts.
- **localStorage** for the inventory, plus JSON export/import so the data is portable.

2. The math

All distances in yards, all offsets in inches. Everything below is flat-fire; do not model shooting angle.

Core relation

```
offset(d) = SH − (d / Z) × (SH + drop(Z)) + drop(d)

  SH       sight height over bore, inches (optic center to bore center)
  Z        desired zero distance, yards
  d        distance the target is actually shot at, yards
  drop(x)  bullet drop below the BORE LINE at x yards, inches (always positive)

  Result is POSITIVE when point of impact is BELOW point of aim.
```

Note the `d/Z` coefficient on the drop term. This is the reason this tool is forgiving: at d=10, Z=100 the entire ballistic model is scaled by 0.1, so a 20% error in drop moves the printed offset by about 0.04 inches. The geometry dominates, not the ballistics.

Drop model

Implement `dropInches(rangeYd, load)` as a single swappable function. A G1 point-mass integrator is preferred if you want it; a constant-velocity vacuum approximation is acceptable and will pass the test vectors below:

```
g = 386.09 in/s²
t = (3 × rangeYd) / v0        // seconds, v0 in ft/s
drop = 0.5 × g × t²
```

Whichever you choose, it must reproduce these **golden vectors** within ±0.03 in. They come from the reference output appended to this document and are the acceptance test for the math:

SH	d	Z	Load	Expected offset
2.6	10	100	5.56, 55gr @ 3000 fps	2.14 in low
2.6	10	25	9mm, 115gr @ 1300 fps	1.41 in low
2.6	10	50	300 BLK, 125gr @ 2175 fps	1.92 in low
2.6	10	50	300 BLK, 220gr @ 1050 fps	1.42 in low

Also verify sensitivity behaves correctly: raising SH from 2.6 to 2.9 must increase the 100-yard-zero offset to 2.41 in, and the 25-yard-zero offset to 1.59 in. If your model does not reproduce the different sensitivity between those two, the `d/Z` term is wrong.

Angular conversions

```
inchesPerMOA(d) = 1.047 × d / 100
inchesPerMIL(d) = 3.600 × d / 100
```

3. Click calculator

This is the feature that makes the tool useful at the bench rather than at the desk. After the user shoots a group, they measure where the group center landed **relative to the printed impact point** — not relative to the aim point — and the app converts that into turret clicks.

Inputs

- Horizontal miss, inches, signed (positive = group is right of the impact point)
- Vertical miss, inches, signed (positive = group is above the impact point)
- Turret click value: 1/4 MOA, 1/2 MOA, 1/3 MOA, 1 MOA, 0.1 mil, 0.05 mil
- Shooting distance is inherited from the target, not re-entered

Output

Correction magnitude in the turret's own units, rounded to whole clicks, with an explicit direction word. Move the impact toward the desired point, so the correction opposes the miss:

```
clicks = round( missInches / inchesPerUnit(d) / clickValue )

  group RIGHT of impact point  →  dial LEFT
  group ABOVE impact point     →  dial DOWN

Render as: "12 clicks LEFT, 5 clicks DOWN"
```

Show the raw angular value alongside the click count ("0.30 in = 2.9 MOA"), because the user needs to sanity-check it against a turret that may not match the selected click value.

Required warning — do not omit this

Compute click resolution at the shooting distance and display it always: `resolution = inchesPerUnit(d) × clickValue`. At 10 yards a 1/4 MOA click moves impact about 0.026 in, which is finer than anyone can read on paper. When resolution is below 0.05 in, show a plain-language caution that the user is now measuring their own group dispersion rather than their zero, and should stop adjusting once the group center is inside the impact circle.

This warning is a correctness feature, not a nicety. Without it the tool actively encourages chasing noise.

4. Inventory

Firearms persist in localStorage. Suggested shape — adapt freely, but keep it JSON-serializable and versioned so a future schema change can migrate:

```
{
  schema: 1,
  guns: [{
    id, name,                 // "AR-15 16in"
    cartridge,                // "5.56 NATO"
    sightHeight,              // inches, default 2.6
    load: { name, v0, bc },   // ft/s, G1
    defaultZero,              // yards
    clickValue                // "0.25moa"
  }]
}
```

Barrel length

Muzzle velocity stays the authoritative input — it is what the math needs, and the velocity-to-length relationship is load-specific and not cleanly a function. Add barrel length as a **convenience** field: when the user picks a preset load

and enters a barrel length, estimate velocity from the preset's reference length and a per-cartridge fps-per-inch figure, write the result into the velocity field, and mark it visibly as an estimate. The velocity field remains directly editable and a user-entered or chronographed value always wins and clears the estimate flag.

Rough figures for the estimator, all approximate and worth labelling as such: 5.56 loses roughly 25–35 fps per inch below 16 in, 300 BLK supersonic roughly 15–20, 300 BLK subsonic almost nothing since it is already near its plateau, and 9mm carbines actually gain velocity with length up to about 16 in. Do not extrapolate a preset far outside its reference range without warning the user.

Keep the magnitude in perspective in the UI copy. A 5.56 SBR at 2650 fps rather than 3000 changes drop at 100 yd from about 2.2 in to 2.8 in, but the d/Z coefficient scales that to roughly 0.06 in on a 10-yard target — well under 1 MOA at the far zero. Support the input, but do not imply it is critical.

Inventory fields

- **Sight height defaults** offered as pickable presets, since most users have not measured theirs: 2.4 in low mount, 2.6 in standard AR flat-top / absolute co-witness, 2.9 in 1.93 in cantilever, 3.1 in tall. Always editable.
- **Preset load library** as an inline JSON constant with typical muzzle velocity and G1 BC: .22 LR, 5.56/.223 (55/62/77gr), .300 BLK supersonic and subsonic, 9mm carbine, .308, 6.5 Creedmoor, 7.62x39, .350 Legend. Every field editable after selection.
- **Batch output.** Select several guns and emit one PDF: a cover/instructions page, then one page per target, exactly as the reference output does.

5. Handguns

The math is platform-agnostic — a handgun is just a much smaller sight height — so support it. But the honest result for most handgun cases is that the tool should tell the user not to bother, and it must say so rather than printing a target that implies precision nobody has.

Sight type	Typical height	Offset at 10 yd for a 25 yd zero
Iron sights	0.60 – 0.75 in	about 0.20 in
Slide-mounted red dot	0.90 – 1.10 in	about 0.42 in
AR with standard optic	2.60 in	about 1.41 in

A slide-mounted dot lands essentially on the 0.4 in threshold from the near-zero-offset rule, and irons fall well below it. Both are also smaller than the shooter's own dispersion with a handgun. The premise of this whole method is that sight height is large relative to the correction being made; on a handgun it is not. So when the computed offset falls below the threshold, emit the single combined mark and state plainly that at this distance point of aim is point of impact.

Sight picture — the one that will produce wrong answers

Iron-sight shooters using a **6 o'clock hold** deliberately aim below the intended impact, and that hold offset stacks on top of the geometric offset this tool computes. A combat or center hold does not. The app must ask which the user runs whenever irons are selected, and add the hold offset explicitly. Assuming center hold silently will give a large fraction of iron-sight users a target that is simply wrong, in a way they are likely to blame on their shooting.

- Separate sight-height presets for handguns — rifle defaults around 2.6 in are meaningless here.
- Note that a 6 o'clock hold offset depends on the target's own size, so it cannot be a fixed number; take it as a user input in inches.

6. Edge cases that will ship broken if you skip them

These are the actual engineering content of this project. The happy path is trivial; these are not.

Negative offset — impact above aim

The reference output hardcodes the assumption that impact is below aim. That is false whenever the shooting distance is past the near crossing of the line of sight. A 100-yard zero shot at 60 yards prints **high**. The layout must detect a negative offset, place the impact grid above the aim diamond, flip the dimension arrow, and relabel to "high". Build this as a signed layout from the start rather than bolting it on, and add a test case for it.

Near-zero offset

When d is at or near the near crossing, the offset approaches zero and the aim and impact marks collide. Below about 0.4 in of separation, do not draw a two-point target. Emit a single combined aim/impact mark and state that at this distance the point of aim is the point of impact.

Offset larger than the paper

A short shooting distance with a long zero produces offsets that exceed the printable area. Validate before drawing. Fall back in this order: shrink the grid → landscape → refuse with a specific message naming the required span and suggesting a longer shooting distance. The aim mark and impact mark must both fit with their labels; never silently clip either off the page edge.

Print scaling — two rulers, not one

A target printed at 95% is worse than no target, because it is confidently wrong. Every page must carry **two** 4-inch rulers: one horizontal along the bottom, one vertical up the left margin. Tick both every 1/16 in, with longer ticks on quarters and whole inches, and number each inch.

The two rulers are not redundant. A laser printer sets horizontal position optically, with a scanning mirror, and vertical position mechanically, by advancing the paper. Those two axes can scale by different amounts. Because every offset this tool produces is vertical, **the vertical ruler is the one that governs correctness** — a horizontal bar reading exactly 4 in proves nothing about the axis that matters. Label the vertical ruler to say so.

This was found the hard way. Mobile print paths (AirPrint, Android/Mopria) silently scale every job to the printer's printable area, typically around 95%, and expose no scale control whatsoever. Desktop viewers default to fit-to-page just as often. Assume the user's print path is hostile.

Calibration input

Provide a calibration field: the user prints one page, measures both rulers, and enters what they actually got. Store per-axis correction factors with the inventory and apply them as independent x and y scale factors on subsequent output. Show the resulting compensation prominently and stamp compensated PDFs with a visible header naming the printer and the factors used, since such a file is wrong on any other printer. Default both factors to 1.0 and make clearing them one click.

Paper size

Letter and A4, both portrait. A4 is narrower and shorter in the relevant dimension; recompute layout from page dimensions rather than hardcoding Letter and hoping.

7. PDF layout specification

The appended reference PDF is the visual target. Match it. jsPDF and reportlab both work in points at 72 per inch with the same built-in Helvetica, so the appended Python source ports nearly line-for-line — read it for exact geometry rather than guessing from the rendering.

Element	Specification
Page	Letter portrait, 0.5 in margins, ONE target per page
Header	Bold 15pt name; 9.5pt subtitle with zero distance, load, sight height; 1.2pt rule
Legend	Top left, how to read the grid; variance table top right
Variance table	7.5pt, offset at SH 2.4 / 2.6 / 2.9 / 3.1, user's value bolded
Grid	Fills the page below the header, centered on the IMPACT point
Grid minor	0.25 in cells, 0.3pt gray 70%
Grid major	1 in lines, 0.6pt gray 45%, with inch numerals along both axes
Grid axes	1.4pt crosshair full width and height through the impact point
Grid border	1.0pt black
Impact circle	1.8pt, r=0.3 in, with a 0.045 in filled center dot
Aim mark	Filled black diamond, 0.55 in half-diagonal, white center pip r=0.05 in
Alignment ticks	Four 0.9pt ticks radiating from 0.62 in to 0.85 in from center
Connector	Dashed 3/3, 0.8pt, between the two marks along the centerline
Dimension arrow	1.6 in left of centerline, arrowheads both ends, bold 11pt label
Text over grid	Every label gets a white knockout rect behind it
Horizontal ruler	4 in along the bottom, ticked every 1/16 in, numbered each inch
Vertical ruler	4 in up the left margin, same ticking, labelled as the governing axis

Draw order matters

Draw the grid **first**, then the impact circle, then the aim diamond, then all labels. The aim mark sits on top of the grid rather than beside it. This is what lets the grid fill the page, and it makes the negative-offset case fall out for free — the diamond simply lands below the impact circle instead of above it, with no layout branch.

Any text that lands on the grid — the dimension label, the two mark labels, the inch numerals — needs a white knockout rectangle drawn behind it, or it becomes unreadable over the grid lines. Suppress an inch numeral entirely when it would fall under the aim diamond; the mirrored numeral on the other side of the axis still gives the reader the scale.

Why the grid fills the page

The grid is the measuring instrument for the click calculator. A user zeroing at 25 or 50 yards can easily put a first group several inches off the impact point, and a small scoring box would leave that group unmeasurable. Size the grid to the largest area that fits below the header, centered on the impact point, and number the inch lines outward from it so any hit reads directly as a signed coordinate that can be typed into the click calculator.

State on the instructions page that a wide grid buys **capture range, not precision**. It does not soften the resolution warning in section 3.

Instructions page

Page one of every generated PDF is a cover with usage steps and an offset summary table for all targets in the document. It must carry these caveats, which are load-bearing rather than boilerplate — see the appended reference for the exact wording:

- Sight height dominates. At short range the offset is nearly pure geometry, so a 0.1 in error in measured sight height shifts the correct offset by roughly 0.06–0.09 in.

- Angular subtension is unforgiving at short range. 1 MOA is only 0.105 in at 10 yards, so a 0.1 in error in group center is about 1 in at 100 yards.
- Range measurement error, quantified per target: compute and print `(SH + drop(Z)) / Z` as inches of offset error per yard of range error. It ranges from negligible on a 100-yard zero to over 0.13 in/yd on short zeros, and users should know which target they are holding.
- Measure BOTH rulers before shooting, and understand that the vertical one is the one that governs the offset.
- This is a first-shot-on-paper zero. Confirm and refine at the real distance.

8. Acceptance criteria

1. Opens from the filesystem with no server and no build step.
2. The four golden vectors reproduce within ±0.03 in.
3. A generated target measured with calipers matches the stated offset exactly.
4. Both 4 in rulers measure 4 in on physical paper at 100% print.
5. The vertical ruler is present, ticked in 1/16 in, and labelled as the governing axis.
6. Calibration factors round-trip: enter a measured value, regenerate, and the printed ruler reads 4 in.
7. A negative-offset case (d past the near crossing) renders correctly flipped and labeled "high".
8. An impossible offset produces a specific refusal message, never a clipped page.
9. Click output gives correct direction and count, verified by hand against a worked example.
10. The low-resolution warning appears at 10 yards with 1/4 MOA clicks.
11. Inventory survives a page reload; export then import round-trips without loss.
12. Entering a barrel length updates velocity, flags it as an estimate, and a manual edit overrides it.
13. A handgun with iron sights returns the combined-mark message, not a two-point target.
14. Selecting irons prompts for hold type, and a 6 o’clock hold changes the computed offset.
15. The grid is centered on the impact point and fills the page below the header.
16. Labels over the grid are legible — white knockout behind every one.
17. Inch numerals hidden by the aim diamond are suppressed, not overdrawn.

Non-goals

No accounts, no cloud sync, no server. No wind, spin drift, Coriolis, or angle-of-fire modeling — at these distances they are noise, and including them would imply a precision this method does not have. No ammunition ordering, no scope database, no images.

Build order

Get the math and a single hardcoded target rendering to PDF first, and verify against the golden vectors before writing any UI. The drawing code is the long pole; the inventory CRUD around it is the easy part and should come last.

Appendix A — Reference output

The following pages are the exact PDF this specification is asking you to reproduce as a parameterized web app. They were generated by the Python in Appendix B.

Do not treat this as a mockup. The geometry is deliberate and has already survived a round of layout-collision fixes — particularly the draw ordering that lets the aim mark sit over a full-page grid, and the numeral suppression under the diamond.

10-Yard Zeroing Offset Targets

Sight height over bore assumed: 2.6 in | Target distance: 10 yards

How to use

1. Print at 100% / Actual Size. Do NOT use "fit to page." Verify with BOTH scale bars on each target page.
2. Hang the correct target at a measured 10 yards. Shoot from a rest — bags, bipod, bench. Not offhand.
   A boresight laser first is fine — use it to get roughly on paper, then let live fire do the actual work.
3. Aim at the center pip of the black diamond for every shot. Never chase the group with your hold.
4. Fire 3–5 rounds. Find the center of the group.
5. Adjust the optic to move the group center onto the circle — not onto the aim point.
6. Confirm with 3 more rounds, then verify at the actual zero distance before trusting it.

Offsets on this sheet

Target	Load assumed	Far zero	POI below POA @ 10 yd
AR-15 / 5.56 NATO	55 gr FMJ @ ~3000 fps	100 yd	2.14 in
AR-9 / 9mm	115 gr @ ~1300 fps	25 yd	1.41 in
300 BLACKOUT / SUPERSONIC	125 gr @ ~2175 fps	50 yd	1.92 in
300 BLACKOUT / SUBSONIC	220 gr @ ~1050 fps	50 yd	1.42 in

Using the grid

One target per page. The grid fills the sheet so a first group that lands well off the mark is still a readable coordinate.
Minor squares are 0.25 in, heavy lines and numerals are 1 in, both measured out from the impact circle.
A wide grid buys capture range, not precision — see the resolution note below before chasing small errors.
Each target page carries a horizontal and a vertical 4 in ruler, both ticked in 1/16 in. Measure both.
Printers can scale differently across the page than along the paper feed. Every offset here is vertical, so the vertical ruler is the one that actually governs your zero — a correct horizontal bar alone proves nothing.

Read this before you trust a 10-yard zero

- Sight height dominates. At 10 yd the offset is almost pure geometry, not ballistics. Every 0.1 in of error in your measured sight height shifts the correct offset by roughly 0.06–0.09 in. Measure optic center to bore center.
- 10 yd is a coarse zero. 1 MOA subtends only 0.105 in at this distance, so a 1/2-MOA click moves impact ~0.05 in — finer than you can reliably read on paper. A 0.1 in error in group center is about 1 in at 100 yd.
- Measure the 10 yards. Each yard of range error shifts the correct offset by about 0.05 in (5.56/100 yd), 0.07 in (300 supersonic), 0.13 in (300 subsonic), 0.13 in (9mm/25 yd). The short-zero targets are the fussy ones.
- Treat this as a first-shot-on-paper zero. It gets you close enough to confirm at real distance, then refine there.
- The 300 BLK supersonic and subsonic targets are not interchangeable — the trajectories differ by ~0.5 in at 10 yd.
- Load velocities are typical 16 in barrel figures. Substituting a very different load mostly matters for the 300 BLK subsonic target; for the others the change at 10 yd is well under 0.05 in.

Formula used

offset = SH − (10 / Z) × (SH + drop@Z) + drop@10

SH = sight height over bore, Z = desired zero distance in yards, drops measured from the bore line.

HORIZONTAL 4.00 in
Both bars must read 4.00 in. If either does not, reprint at 100% / Actual Size.

AR-15 / 5.56 NATO

Zero on this target at 10 yd → zeroed at 100 yd | 55 gr FMJ @ ~3000 fps | sight height 2.6 in

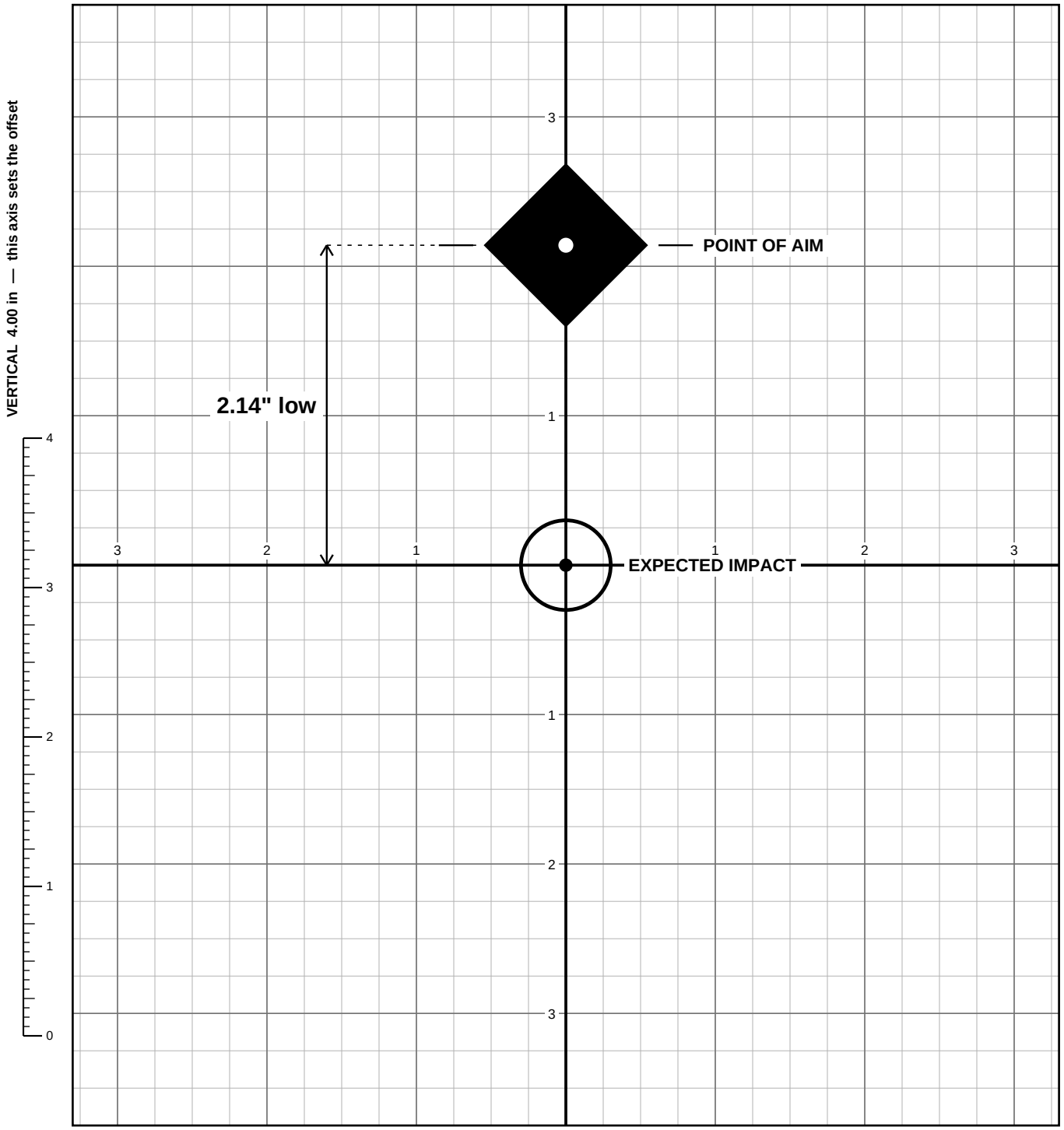
AIM at the diamond. ADJUST until groups center on the circle.

Grid squares = 0.25 in. Heavy lines and numerals = 1 in.

Measure the group center from the circle to get your click correction.

IF YOUR SIGHT HEIGHT DIFFERS:

sight ht	POI below POA
2.4 in	1.96 in
2.6 in	**2.14 in <—**
2.9 in	2.41 in
3.1 in	2.59 in



HORIZONTAL 4.00 in

Both bars must read 4.00 in. If either does not, reprint at 100% / Actual Size.

AR-9 / 9mm

Zero on this target at 10 yd → zeroed at 25 yd | 115 gr @ ~1300 fps | sight height 2.6 in

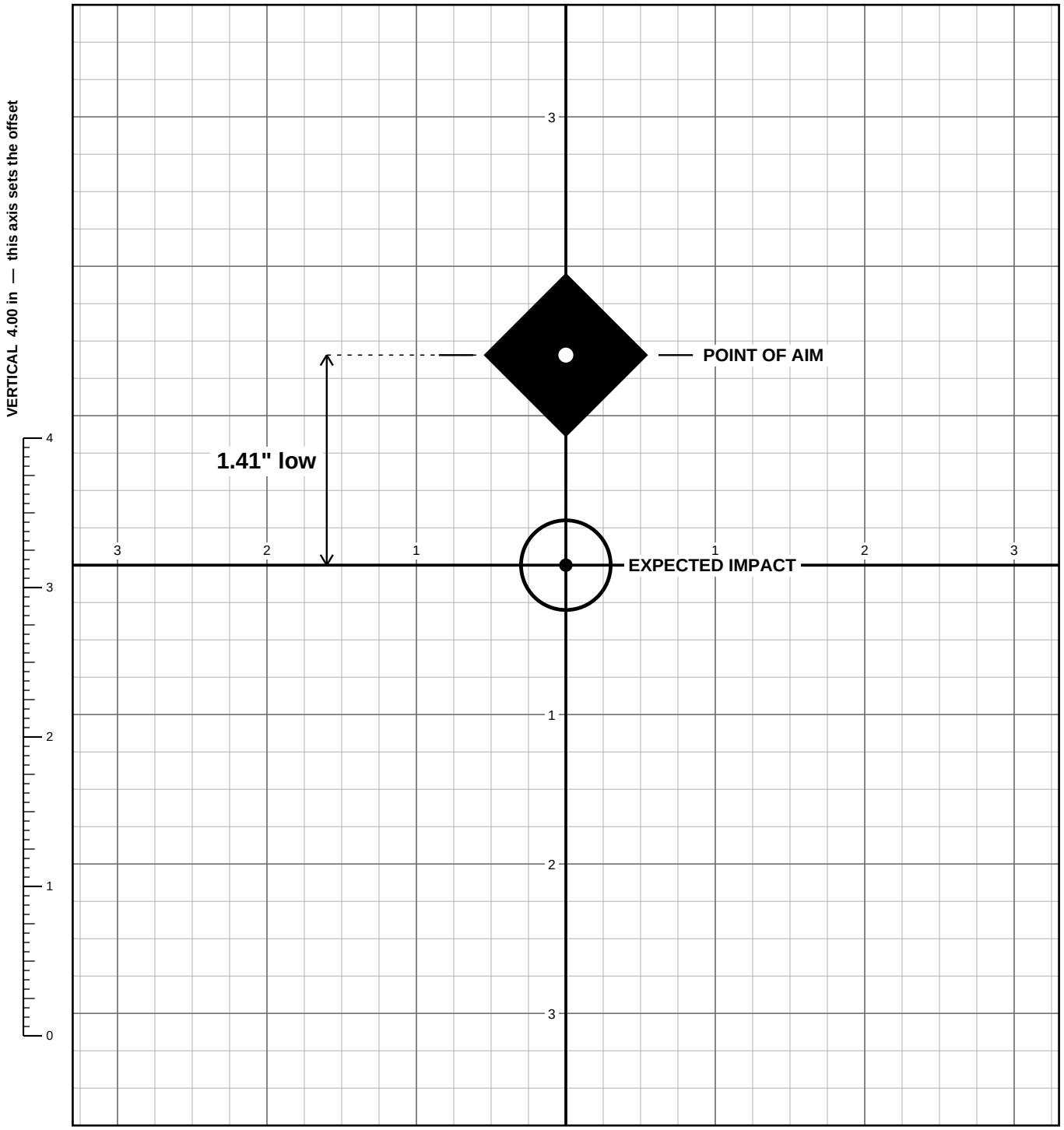
AIM at the diamond. ADJUST until groups center on the circle.

Grid squares = 0.25 in. Heavy lines and numerals = 1 in.

Measure the group center from the circle to get your click correction.

IF YOUR SIGHT HEIGHT DIFFERS:

sight ht	POI below POA
2.4 in	1.29 in
2.6 in	**1.41 in <—**
2.9 in	1.59 in
3.1 in	1.71 in



HORIZONTAL 4.00 in

Both bars must read 4.00 in. If either does not, reprint at 100% / Actual Size.

300 BLACKOUT / SUPERSONIC

Zero on this target at 10 yd → zeroed at 50 yd | 125 gr @ ~2175 fps | sight height 2.6 in

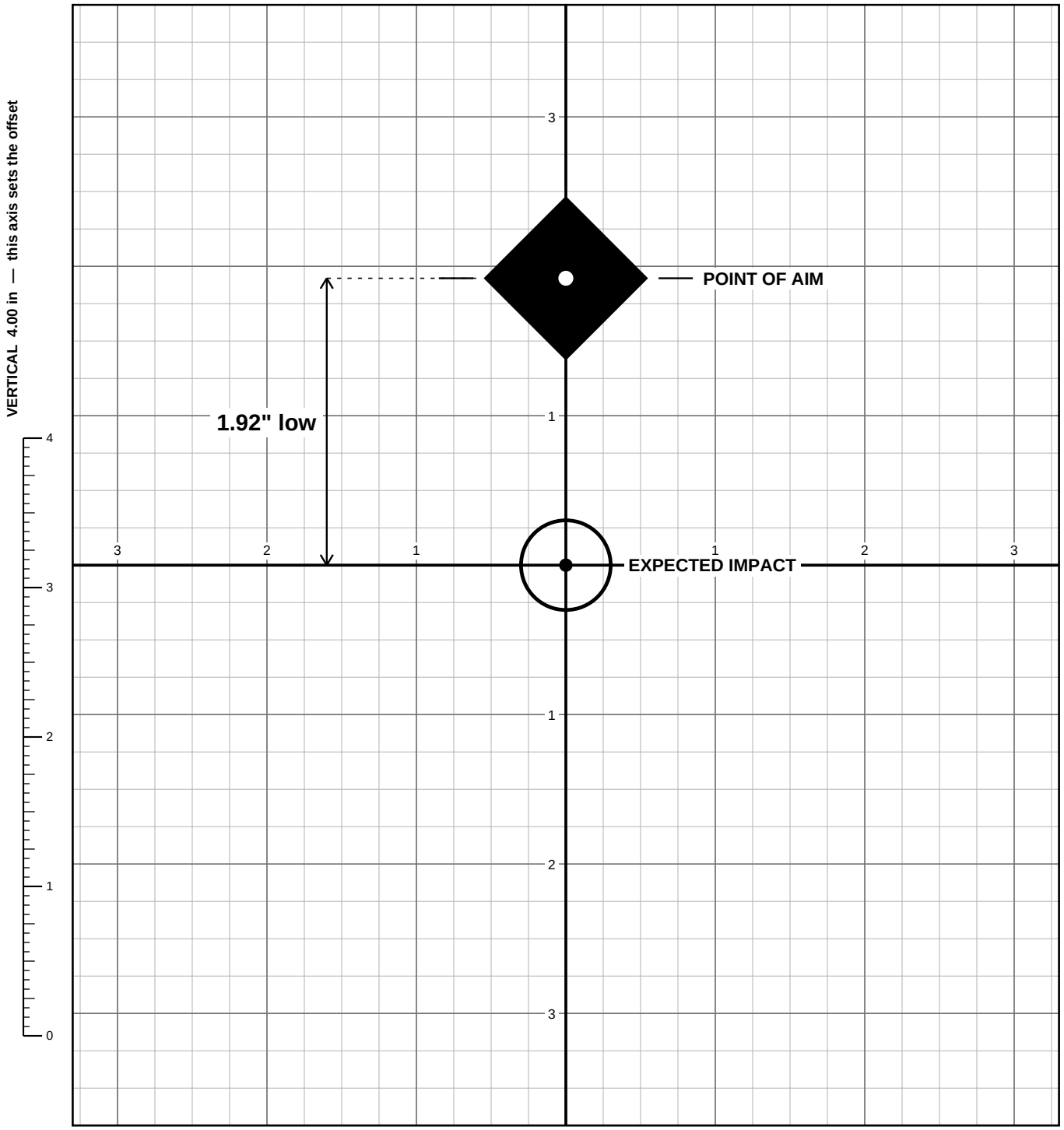
AIM at the diamond. ADJUST until groups center on the circle.

Grid squares = 0.25 in. Heavy lines and numerals = 1 in.

Measure the group center from the circle to get your click correction.

IF YOUR SIGHT HEIGHT DIFFERS:

sight ht	POI below POA
2.4 in	1.76 in
2.6 in	**1.92 in <—**
2.9 in	2.16 in
3.1 in	2.32 in



HORIZONTAL 4.00 in

Both bars must read 4.00 in. If either does not, reprint at 100% / Actual Size.

300 BLACKOUT / SUBSONIC

Zero on this target at 10 yd → zeroed at 50 yd | 220 gr @ ~1050 fps | sight height 2.6 in

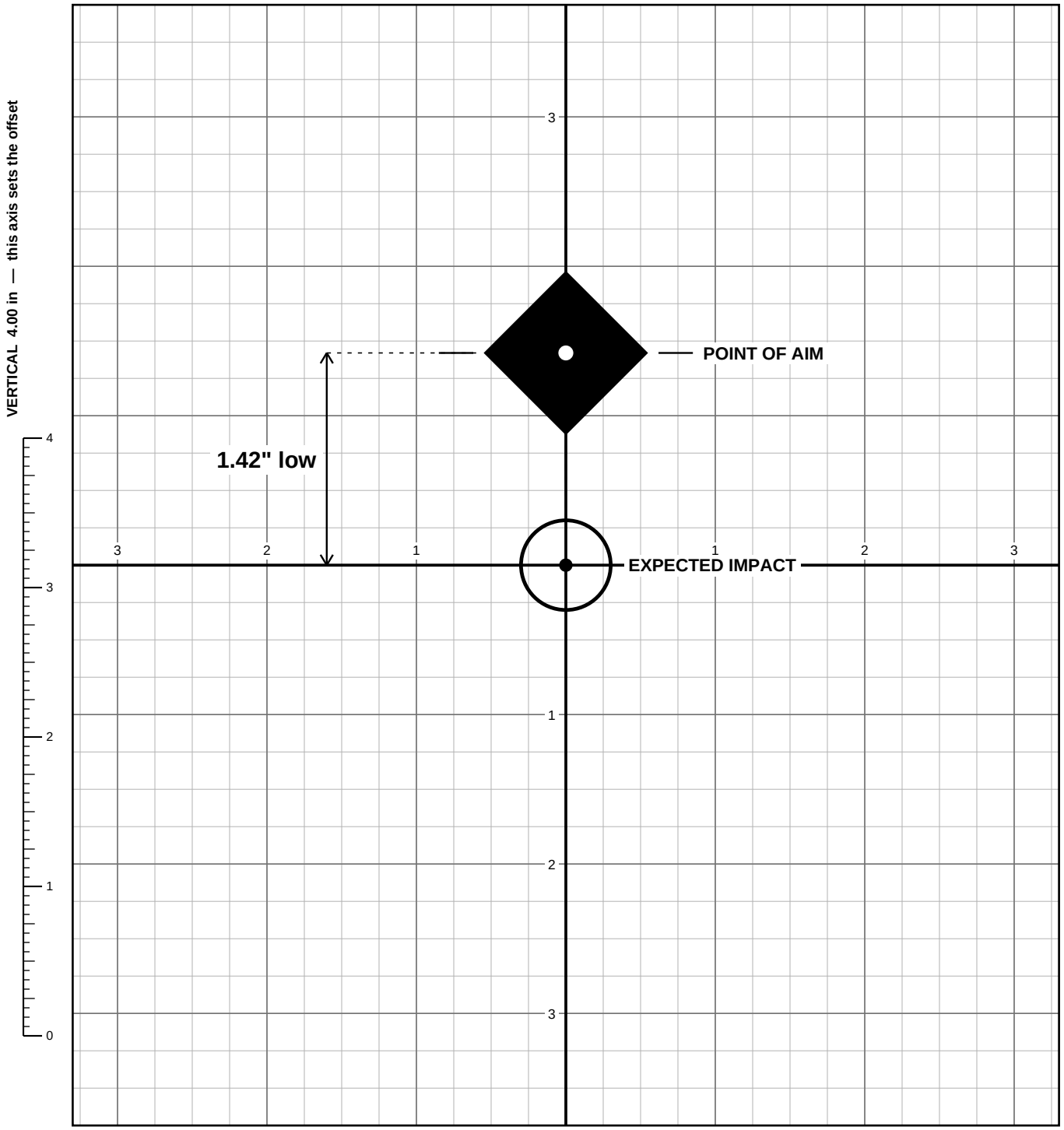
AIM at the diamond. ADJUST until groups center on the circle.

Grid squares = 0.25 in. Heavy lines and numerals = 1 in.

Measure the group center from the circle to get your click correction.

IF YOUR SIGHT HEIGHT DIFFERS:

sight ht	POI below POA
2.4 in	1.26 in
2.6 in	**1.42 in <—**
2.9 in	1.66 in
3.1 in	1.82 in



HORIZONTAL 4.00 in

Both bars must read 4.00 in. If either does not, reprint at 100% / Actual Size.

Appendix B — Reference implementation

The reportlab source that produced Appendix A. Port the drawing geometry from this rather than re-deriving it from the rendering. Both libraries use 72 points per inch, origin at bottom-left, and the same built-in Helvetica metrics, so coordinates transfer directly; jsPDF's y-axis runs downward from the top, so invert y and keep everything else.

```
#!/usr/bin/env python3
"""10-yard zeroing offset targets. One target per page, full-page measurement grid."""
from reportlab.lib.pagesizes import letter
from reportlab.pdfgen import canvas

IN = 72.0
W, H = letter
MARGIN = 0.5 * IN
HOB = 2.6  # sight height over bore, inches

# name, cartridge/load, far zero (yd), drop at far zero (in), drop at 10 yd (in)
TARGETS = [
    ("AR-15  /  5.56 NATO", "55 gr FMJ @ ~3000 fps", 100, 2.200, 0.021),
    ("AR-9  /  9mm", "115 gr @ ~1300 fps", 25, 0.643, 0.103),
    ("300 BLACKOUT  /  SUPERSONIC", "125 gr @ ~2175 fps", 50, 0.985, 0.037),
    ("300 BLACKOUT  /  SUBSONIC", "220 gr @ ~1050 fps", 50, 4.094, 0.159),
]
HOB_TABLE = [2.4, 2.6, 2.9, 3.1]

# grid geometry
CX = W / 2
IMP_Y = 5.15 * IN
GW = 3.30 * IN
GH = 3.75 * IN


def offset(hob, z, drop_z, drop_10, d=10.0):
    """Inches the POI sits below the POA at distance d. Negative means POI is high."""
    return hob - (d / z) * (hob + drop_z) + drop_10


def knockout(c, x, y, text, font="Helvetica", size=8, align="left", pad=2):
    """Draw text with a white box behind it so it stays legible over the grid."""
    w = c.stringWidth(text, font, size)
    if align == "right":
        x0 = x - w
    elif align == "center":
        x0 = x - w / 2.0
    else:
        x0 = x
    c.setFillColorRGB(1, 1, 1)
    c.rect(x0 - pad, y - pad - 0.5, w + 2 * pad, size + pad, stroke=0, fill=1)
    c.setFillColorRGB(0, 0, 0)
    c.setFont(font, size)
    c.drawString(x0, y, text)


SIXTEENTH = IN / 16.0


def _tick_len(i):
    """i counts sixteenths. Longer ticks on quarter and whole inches."""
    if i % 16 == 0:
        return 9.0
    if i % 4 == 0:
        return 5.5
    return 3.0


def scale_bar_h(c, x, y):
    """Horizontal 4 in ruler, ticks upward. Checks the scan-direction axis."""
    c.setStrokeColorRGB(0, 0, 0)
    c.setLineWidth(0.8)
    c.line(x, y, x + 4 * IN, y)
    for i in range(65):
        c.setLineWidth(0.8 if i % 16 == 0 else 0.5)
        xx = x + i * SIXTEENTH
        c.line(xx, y, xx, y + _tick_len(i))
    c.setFillColorRGB(0, 0, 0)
    c.setFont("Helvetica", 6)
    for i in range(5):
        c.drawCentredString(x + i * IN, y + 11, str(i))
    c.setFont("Helvetica-Bold", 7)
    c.drawString(x + 4 * IN + 8, y + 4, "HORIZONTAL  4.00 in")
    c.setFont("Helvetica", 7)
```

```
    c.drawString(x + 4 * IN + 8, y - 5,
                 "Both bars must read 4.00 in. If either does not, reprint at 100% / Actual Size.")


def scale_bar_v(c, x, y):
    """Vertical 4 in ruler, ticks to the right. Checks the paper-feed axis,
    which is the one that governs the aim-to-impact offset."""
    c.setStrokeColorRGB(0, 0, 0)
    c.setLineWidth(0.8)
    c.line(x, y, x, y + 4 * IN)
    for i in range(65):
        c.setLineWidth(0.8 if i % 16 == 0 else 0.5)
        yy = y + i * SIXTEENTH
        c.line(x, yy, x + _tick_len(i), yy)
    c.setFillColorRGB(0, 0, 0)
    c.setFont("Helvetica", 6)
    for i in range(5):
        c.drawString(x + 11, y + i * IN - 2, str(i))
    c.saveState()
    c.translate(x - 3, y + 4 * IN + 10)
    c.rotate(90)
    c.setFont("Helvetica-Bold", 7)
    c.drawString(0, 0, "VERTICAL  4.00 in  \u2014  this axis sets the offset")
    c.restoreState()


def scale_bar(c, x, y):
    scale_bar_h(c, x, y)


def draw_grid(c, aim_y=None):
    """Full-page measurement grid centered on the impact point."""
    minor = 0.25 * IN
    c.setStrokeColorRGB(0.70, 0.70, 0.70)
    c.setLineWidth(0.3)
    i = 1
    while i * minor <= GW:
        c.line(CX + i * minor, IMP_Y - GH, CX + i * minor, IMP_Y + GH)
        c.line(CX - i * minor, IMP_Y - GH, CX - i * minor, IMP_Y + GH)
        i += 1
    i = 1
    while i * minor <= GH:
        c.line(CX - GW, IMP_Y + i * minor, CX + GW, IMP_Y + i * minor)
        c.line(CX - GW, IMP_Y - i * minor, CX + GW, IMP_Y - i * minor)
        i += 1

    c.setStrokeColorRGB(0.45, 0.45, 0.45)
    c.setLineWidth(0.6)
    i = 1
    while i * IN <= GW:
        c.line(CX + i * IN, IMP_Y - GH, CX + i * IN, IMP_Y + GH)
        c.line(CX - i * IN, IMP_Y - GH, CX - i * IN, IMP_Y + GH)
        i += 1
    i = 1
    while i * IN <= GH:
        c.line(CX - GW, IMP_Y + i * IN, CX + GW, IMP_Y + i * IN)
        c.line(CX - GW, IMP_Y - i * IN, CX + GW, IMP_Y - i * IN)
        i += 1

    c.setStrokeColorRGB(0, 0, 0)
    c.setLineWidth(1.0)
    c.rect(CX - GW, IMP_Y - GH, 2 * GW, 2 * GH, stroke=1, fill=0)
    c.setLineWidth(1.4)
    c.line(CX - GW, IMP_Y, CX + GW, IMP_Y)
    c.line(CX, IMP_Y - GH, CX, IMP_Y + GH)

    for i in range(1, int(GW / IN) + 1):
        for sx in (1, -1):
            knockout(c, CX + sx * i * IN, IMP_Y + 5, str(i), size=6.5, align="center", pad=1.5)
    for i in range(1, int(GH / IN) + 1):
        for sy in (1, -1):
            yy = IMP_Y + sy * i * IN
            if aim_y is not None and abs(yy - aim_y) < 0.7 * IN:
                continue  # would be buried under the aim diamond
            knockout(c, CX - 5, yy - 2.5, str(i), size=6.5, align="right", pad=1.5)


def draw_target(c, spec):
    name, load, z, drop_z, drop_10 = spec
    off = offset(HOB, z, drop_z, drop_10)
    top = H - MARGIN

    c.setFillColorRGB(0, 0, 0)
    c.setFont("Helvetica-Bold", 15)
    c.drawString(MARGIN, top - 15, name)
```

```
    c.setFont("Helvetica", 9.5)
    c.drawString(MARGIN, top - 29,
                 "Zero on this target at 10 yd  \u2192  zeroed at %d yd   |   %s   |   sight height %.1f in"
                 % (z, load, HOB))
    c.setLineWidth(1.2)
    c.line(MARGIN, top - 36, W - MARGIN, top - 36)

    tx, ty = W - MARGIN - 2.05 * IN, top - 55
    c.setFont("Helvetica-Bold", 7.5)
    c.drawString(tx, ty + 12, "IF YOUR SIGHT HEIGHT DIFFERS:")
    c.setFont("Helvetica", 7.5)
    c.drawString(tx, ty, "sight ht")
    c.drawString(tx + 0.75 * IN, ty, "POI below POA")
    c.setLineWidth(0.4)
    c.line(tx, ty - 3, tx + 2.0 * IN, ty - 3)
    for i, h in enumerate(HOB_TABLE):
        yy = ty - 12 - i * 10
        bold = abs(h - HOB) < 0.01
        c.setFont("Helvetica-Bold" if bold else "Helvetica", 7.5)
        c.drawString(tx, yy, "%.1f in" % h)
        c.drawString(tx + 0.75 * IN, yy,
                     "%.2f in%s" % (offset(h, z, drop_z, drop_10), "   <\u2014" if bold else ""))

    lx, ly = MARGIN, top - 55
    c.setFillColorRGB(0, 0, 0)
    c.setFont("Helvetica-Bold", 8)
    c.drawString(lx, ly, "AIM at the diamond.  ADJUST until groups center on the circle.")
    c.setFont("Helvetica", 7.5)
    c.drawString(lx, ly - 11, "Grid squares = 0.25 in.  Heavy lines and numerals = 1 in.")
    c.drawString(lx, ly - 21, "Measure the group center from the circle to get your click correction.")

    aim_y = IMP_Y + off * IN
    draw_grid(c, aim_y)

    c.setStrokeColorRGB(0, 0, 0)
    c.setDash(3, 3)
    c.setLineWidth(0.8)
    c.line(CX, aim_y - 0.58 * IN, CX, IMP_Y + 0.32 * IN)
    c.setDash()

    c.setLineWidth(1.8)
    c.circle(CX, IMP_Y, 0.3 * IN, stroke=1, fill=0)
    c.setFillColorRGB(0, 0, 0)
    c.circle(CX, IMP_Y, 0.045 * IN, stroke=0, fill=1)

    r = 0.55 * IN
    p = c.beginPath()
    p.moveTo(CX, aim_y + r)
    p.lineTo(CX + r, aim_y)
    p.lineTo(CX, aim_y - r)
    p.lineTo(CX - r, aim_y)
    p.close()
    c.setFillColorRGB(0, 0, 0)
    c.drawPath(p, stroke=0, fill=1)
    c.setFillColorRGB(1, 1, 1)
    c.circle(CX, aim_y, 0.05 * IN, stroke=0, fill=1)

    c.setStrokeColorRGB(0, 0, 0)
    c.setLineWidth(0.9)
    for dx, dy in ((1, 0), (-1, 0), (0, 1), (0, -1)):
        c.line(CX + dx * 0.62 * IN, aim_y + dy * 0.62 * IN,
               CX + dx * 0.85 * IN, aim_y + dy * 0.85 * IN)

    ax = CX - 1.6 * IN
    c.setLineWidth(0.9)
    c.line(ax, aim_y, ax, IMP_Y)
    for yy, d in ((aim_y, -1), (IMP_Y, 1)):
        c.line(ax, yy, ax - 3, yy + d * 5)
        c.line(ax, yy, ax + 3, yy + d * 5)
    c.setDash(2, 3)
    c.setLineWidth(0.6)
    c.line(ax, aim_y, CX - 0.6 * IN, aim_y)
    c.line(ax, IMP_Y, CX - 0.32 * IN, IMP_Y)
    c.setDash()
    knockout(c, ax - 5, (aim_y + IMP_Y) / 2 - 4, '%.2f" low' % off,
             font="Helvetica-Bold", size=11, align="right", pad=3)

    knockout(c, CX + 0.92 * IN, aim_y - 3, "POINT OF AIM", font="Helvetica-Bold", size=8.5)
    knockout(c, CX + 0.42 * IN, IMP_Y - 3, "EXPECTED IMPACT", font="Helvetica-Bold", size=8.5)

    scale_bar_h(c, MARGIN, 0.55 * IN)
    scale_bar_v(c, MARGIN + 0.12 * IN, 2.0 * IN)
    c.showPage()
def instructions_page(c):
    c.setFont("Helvetica-Bold", 18)
```

```
c.drawString(MARGIN, H - 0.85 * IN, "10-Yard Zeroing Offset Targets")
c.setFont("Helvetica", 10)
c.drawString(MARGIN, H - 1.1 * IN,
             "Sight height over bore assumed: %.1f in    |    Target distance: 10 yards" % HOB)
c.setLineWidth(1.2)
c.line(MARGIN, H - 1.2 * IN, W - MARGIN, H - 1.2 * IN)

y = H - 1.55 * IN
c.setFont("Helvetica-Bold", 11)
c.drawString(MARGIN, y, "How to use")
y -= 16
steps = [
    "1.  Print at 100% / Actual Size. Do NOT use \"fit to page.\" Verify with BOTH scale bars on each
        target page.",
    "2.  Hang the correct target at a measured 10 yards. Shoot from a rest \u2014 bags, bipod, bench.
        Not offhand.",
    "     A boresight laser first is fine \u2014 use it to get roughly on paper, then let live fire do
        the actual work.",
    "3.  Aim at the center pip of the black diamond for every shot. Never chase the group with your
        hold.",
    "4.  Fire 3\u20135 rounds. Find the center of the group.",
    "5.  Adjust the optic to move the group center onto the circle \u2014 not onto the aim point.",
    "6.  Confirm with 3 more rounds, then verify at the actual zero distance before trusting it.",
]
c.setFont("Helvetica", 9.5)
for s in steps:
    c.drawString(MARGIN, y, s)
    y -= 14

y -= 10
c.setFont("Helvetica-Bold", 11)
c.drawString(MARGIN, y, "Offsets on this sheet")
y -= 6
rows = [("Target", "Load assumed", "Far zero", "POI below POA @ 10 yd")]
for t in TARGETS:
    rows.append((t[0].replace("  /  ", " / "), t[1], "%d yd" % t[2],
                 "%.2f in" % offset(HOB, t[2], t[3], t[4])))
colx = [MARGIN, MARGIN + 1.95 * IN, MARGIN + 3.9 * IN, MARGIN + 4.85 * IN]
for ri, row in enumerate(rows):
    y -= 15
    c.setFont("Helvetica-Bold" if ri == 0 else "Helvetica", 9)
    for ci, cell in enumerate(row):
        c.drawString(colx[ci], y, cell)
    if ri == 0:
        c.setLineWidth(0.5)
        c.line(MARGIN, y - 4, W - MARGIN, y - 4)

y -= 28
c.setFont("Helvetica-Bold", 11)
c.drawString(MARGIN, y, "Using the grid")
y -= 15
gridnotes = [
    "One target per page. The grid fills the sheet so a first group that lands well off the mark is
        still a readable coordinate.",
    "Minor squares are 0.25 in, heavy lines and numerals are 1 in, both measured out from the impact
        circle.",
    "A wide grid buys capture range, not precision \u2014 see the resolution note below before chasing
        small errors.",
    "Each target page carries a horizontal and a vertical 4 in ruler, both ticked in 1/16 in. Measure
        both.",
    "Printers can scale differently across the page than along the paper feed. Every offset here is
        vertical, so the",
    "vertical ruler is the one that actually governs your zero \u2014 a correct horizontal bar alone
        proves nothing.",
]
c.setFont("Helvetica", 9)
for n in gridnotes:
    c.drawString(MARGIN, y, n)
    y -= 13

y -= 12
c.setFont("Helvetica-Bold", 11)
c.drawString(MARGIN, y, "Read this before you trust a 10-yard zero")
y -= 16
notes = [
    "\u2022  Sight height dominates. At 10 yd the offset is almost pure geometry, not ballistics. Every
        0.1 in of error in your",
    "    measured sight height shifts the correct offset by roughly 0.06\u20130.09 in. Measure optic
        center to bore center.",
    "\u2022  10 yd is a coarse zero. 1 MOA subtends only 0.105 in at this distance, so a 1/2-MOA click
        moves impact ~0.05 in \u2014",
    "    finer than you can reliably read on paper. A 0.1 in error in group center is about 1 in at 100
        yd.",
    "\u2022  Measure the 10 yards. Each yard of range error shifts the correct offset by about 0.05 in
        (5.56/100 yd),",
```

```
        "    0.07 in (300 supersonic), 0.13 in (300 subsonic), 0.13 in (9mm/25 yd). The short-zero targets
            are the fussy ones.",
        "\u2022  Treat this as a first-shot-on-paper zero. It gets you close enough to confirm at real
            distance, then refine there.",
        "\u2022  The 300 BLK supersonic and subsonic targets are not interchangeable \u2014 the trajectories
            differ by ~0.5 in at 10 yd.",
        "\u2022  Load velocities are typical 16 in barrel figures. Substituting a very different load mostly
            matters for the 300 BLK",
        "    subsonic target; for the others the change at 10 yd is well under 0.05 in.",
    ]
    c.setFont("Helvetica", 9)
    for n in notes:
        c.drawString(MARGIN, y, n)
        y -= 13

    y -= 12
    c.setFont("Helvetica-Bold", 9)
    c.drawString(MARGIN, y, "Formula used")
    y -= 13
    c.setFont("Helvetica", 9)
    c.drawString(MARGIN, y, "offset = SH \u2212 (10 / Z) \u00d7 (SH + drop@Z) + drop@10")
    y -= 12
    c.setFont("Helvetica-Oblique", 8)
    c.drawString(MARGIN, y,
                 "SH = sight height over bore, Z = desired zero distance in yards, drops measured from the
                     bore line.")

    scale_bar(c, MARGIN, 0.6 * IN)
    c.showPage()


def main():
    c = canvas.Canvas("/mnt/user-data/outputs/10yd-zero-targets.pdf", pagesize=letter)
    c.setTitle("10-Yard Zeroing Offset Targets")
    instructions_page(c)
    for spec in TARGETS:
        draw_target(c, spec)
    c.save()
    print("written")


if __name__ == "__main__":
    main()
```